WEEK 11 – DATABASE AUTOMATION USING VIEWS AND PROCEDURES

1. Introduction

Database automation helps reduce repetitive SQL coding and improves the efficiency of database operations. In an e-commerce database, operations such as order processing, sales calculation, customer analysis, and report generation are performed frequently.

To simplify these operations, database features such as Views, Stored Procedures, and Functions are used. These features help organize queries, automate repeated tasks, and perform calculations efficiently.

In this practical, database automation techniques are implemented in the existing ArtHub database system.

2. Objectives

- To create views for frequently used reports.
- To develop stored procedures for order processing.
- To create functions for sales calculations.
- To automate repetitive database operations.
- To test database objects and verify their results.

3. Create Views for Frequently Used Reports

A View is a virtual table created using one or more SQL queries. It stores the query definition and provides simplified access to frequently used information.

Views are useful for displaying commonly required information without writing the same query repeatedly.

The following views are created:

3.1 Customer Order Report View

This view displays customer order information, including customer name, order ID, order date, total amount, and order status. It helps retrieve customer order details in a single report.

```
SQL> CREATE OR REPLACE VIEW CUSTOMER_ORDER_REPORT AS SELECT C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS FROM Customer C JOIN O
rders O ON C.CUSTOMER_ID = O.CUSTOMER_ID;

View created.

SQL> SELECT * FROM CUSTOMER_ORDER_REPORT;

CUSTOMER_NAME                  ORDER_ID   ORDER_DATE         TOTAL_AMOUNT
------------------------------ ---------- ------------------ ------------
ORDER_STATUS
--------------------
Dhoni                          O1         20-SEP-26                  5000
Delivered

Virat                          O2         21-SEP-26                  5200
Shipped

Ashwin                         O3         21-SEP-26                  1800
Delivered


CUSTOMER_NAME                  ORDER_ID   ORDER_DATE         TOTAL_AMOUNT
------------------------------ ---------- ------------------ ------------
ORDER_STATUS
--------------------
SanjuSamson                    O4         22-SEP-26                  2200
Processing

Shreyas Iyer                   O5         22-SEP-26                  3500
Shipped

Hardhik Pandya                 O6         23-SEP-26                  4000
Delivered


CUSTOMER_NAME                  ORDER_ID   ORDER_DATE         TOTAL_AMOUNT
------------------------------ ---------- ------------------ ------------
ORDER_STATUS
--------------------
Ruturaj Gaikwad                O7         24-SEP-26                  2800
Processing
```

3.2 Product Sales Report View

This view displays product name, quantity sold, and total revenue. It helps analyse the sales performance of artworks in the ArtHub system.

```
SQL> CREATE OR REPLACE VIEW PRODUCT_SALES_REPORT AS SELECT A.ARTWORK_NAME, NVL(SUM(D.QUANTITY),0) AS TOTAL_QUANTITY_SOLD, NVL(SUM(D.QUANTITY * D.PRICE),0) A
S TOTAL_REVENUE FROM ArtProduct A LEFT JOIN OrderDetails D ON A.ARTWORK_ID = D.ARTWORK_ID GROUP BY A.ARTWORK_ID, A.ARTWORK_NAME;

View created.

SQL> SELECT * FROM PRODUCT_SALES_REPORT;

ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
Handmade Craft
                  0             0

Charcoal Dreams
                  1          2200

Colors of Nature
                  1          5200


ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
The Thinker Sculpture
                  0             0

Future City
                  1          4000

Sunset Over Hills
                  1          5000


ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
Pencil Portrait
```

```
ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
The Thinker Sculpture
                  0             0

Future City
                  1          4000

Sunset Over Hills
                  1          5000


ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
Pencil Portrait
                  1          1800

Digital Galaxy
                  1          3500

Abstract Dreams
                  0             0


ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
Morning Light
                  0             0


10 rows selected.
```

3.3 Payment Status View

This view displays order ID, payment mode, payment status, and transaction amount. It helps monitor payment details and identify the payment status of each order.

```
SQL> CREATE OR REPLACE VIEW PAYMENT_STATUS_REPORT AS SELECT ORDER_ID, PAYMENT_MODE, PAYMENT_STATUS, TRANSACTION_AMOUNT FROM Payment;

View created.

SQL> SELECT * FROM PAYMENT_STATUS_REPORT;

ORDER_ID   PAYMENT_MODE         PAYMENT_STATUS       TRANSACTION_AMOUNT
---------- -------------------- -------------------- ------------------
O1         UPI                  Completed                          5000
O2         Card                 Completed                          5200
O3         UPI                  Completed                          1800
O4         Net Banking          Pending                            2200
O6         UPI                  Completed                          4000
O5         Card                 Completed                          3500
O7         Card                 Pending                            2800

7 rows selected.
```

4. Develop Stored Procedures for Order Processing

A Stored Procedure is a collection of SQL statements stored in the database that can be executed whenever required. It helps automate repeated database operations and reduces the need to write the same SQL statements repeatedly.

The following stored procedures are developed:

4.1 Add New Customer Order

This procedure is used to process a new customer order. It performs the following operations:

- Insert order details.
- Add purchased products.
- Calculate the total amount.

```
SQL> CREATE OR REPLACE PROCEDURE ADD_NEW_ORDER (P_ORDER_ID VARCHAR2, P_CUSTOMER_ID NUMBER, P_ARTWORK_ID VARCHAR2, P_QUANTITY NUMBER) AS V_PRICE NUMBER(10,2)
; V_TOTAL NUMBER(10,2); BEGIN SELECT PRICE INTO V_PRICE FROM ArtProduct WHERE ARTWORK_ID = P_ARTWORK_ID; V_TOTAL := V_PRICE * P_QUANTITY; INSERT INTO Orders
 (ORDER_ID, CUSTOMER_ID, ORDER_DATE, TOTAL_AMOUNT, ORDER_STATUS) VALUES (P_ORDER_ID, P_CUSTOMER_ID, SYSDATE, V_TOTAL, 'PENDING'); INSERT INTO OrderDetails (
ORDER_DETAIL_ID, ORDER_ID, ARTWORK_ID, QUANTITY, PRICE) VALUES ('OD'||P_ORDER_ID, P_ORDER_ID, TO_NUMBER(P_ARTWORK_ID), P_QUANTITY, V_PRICE); END;
  2  /

Procedure created.

SQL> EXEC ADD_NEW_ORDER('O8', 8, '101', 1);

PL/SQL procedure successfully completed.

SQL> SELECT * FROM Orders WHERE ORDER_ID = 'O8';

ORDER_ID   CUSTOMER_ID ORDER_DATE         TOTAL_AMOUNT ORDER_STATUS
---------- ----------- ----------------- ------------ --------------------
O8                   8 11-NOV-26                 5000 PENDING

SQL> SELECT * FROM OrderDetails WHERE ORDER_ID = 'O8';

ORDER_DETA ORDER_ID   ARTWORK_ID   QUANTITY      PRICE
---------- ---------- ---------- ---------- ----------
OD08       O8         101                 1       5000
```

4.2 Update Order Status

This procedure is used to update the status of an order during the order processing stages. The order status can be changed from Pending to Shipped and then to Delivered.

```
SQL> CREATE OR REPLACE PROCEDURE UPDATE_ORDER_STATUS (P_ORDER_ID VARCHAR2, P_STATUS VARCHAR2) AS BEGIN UPDATE Orders SET ORDER_STATUS = P_STATUS WHERE ORDER
_ID = P_ORDER_ID; END;
  2  /

Procedure created.

SQL> EXEC UPDATE_ORDER_STATUS('O8', 'SHIPPED');

PL/SQL procedure successfully completed.

SQL> SELECT ORDER_ID, ORDER_STATUS FROM Orders WHERE ORDER_ID = 'O8';

ORDER_ID   ORDER_STATUS
---------- --------------------
O8         SHIPPED
```

4.3 Generate Customer Purchase History

This procedure accepts Customer_ID as input and retrieves the customer's order details, purchase amount, and order status. It helps maintain and analyse customer purchase history.

```
SQL> CREATE OR REPLACE PROCEDURE CUSTOMER_PURCHASE_HISTORY (P_CUSTOMER_ID NUMBER) AS BEGIN FOR R IN (SELECT C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, O.TOT
AL_AMOUNT, O.ORDER_STATUS FROM Customer C JOIN Orders O ON C.CUSTOMER_ID = O.CUSTOMER_ID WHERE C.CUSTOMER_ID = P_CUSTOMER_ID) LOOP DBMS_OUTPUT.PUT_LINE('CUS
TOMER NAME: ' || R.CUSTOMER_NAME); DBMS_OUTPUT.PUT_LINE('ORDER ID: ' || R.ORDER_ID); DBMS_OUTPUT.PUT_LINE('ORDER DATE: ' || R.ORDER_DATE); DBMS_OUTPUT.PUT_L
INE('TOTAL AMOUNT: ' || R.TOTAL_AMOUNT); DBMS_OUTPUT.PUT_LINE('ORDER STATUS: ' || R.ORDER_STATUS); DBMS_OUTPUT.PUT_LINE('-------------------------'); END LO
OP; END;
  2  /

Procedure created.

SQL> SET SERVEROUTPUT ON;
SQL> EXEC CUSTOMER_PURCHASE_HISTORY(1);
CUSTOMER NAME: Dhoni
ORDER ID: O1
ORDER DATE: 20-SEP-26
TOTAL AMOUNT: 5000
ORDER STATUS: Delivered
-------------------------

PL/SQL procedure successfully completed.
```

5. Create Functions for Sales Calculations

A Function performs a specific calculation and returns a value. Functions help perform frequently required calculations in the database.

The following functions are created:

5.1 Calculate Total Sales

This function accepts a date range as input and returns the total revenue generated during that period. It helps analyse sales performance over a selected period.

```
SQL> CREATE OR REPLACE FUNCTION CALCULATE_TOTAL_SALES (P_START_DATE DATE, P_END_DATE DATE) RETURN NUMBER AS V_TOTAL NUMBER(10,2); BEGIN SELECT NVL(SUM(TOTAL
_AMOUNT),0) INTO V_TOTAL FROM Orders WHERE ORDER_DATE >= P_START_DATE AND ORDER_DATE < P_END_DATE + 1; RETURN V_TOTAL; END;
  2  /

Function created.

SQL> SELECT CALCULATE_TOTAL_SALES(TO_DATE('20-09-2026','DD-MM-YYYY'), TO_DATE('24-09-2026','DD-MM-YYYY')) AS TOTAL_SALES FROM DUAL;

TOTAL_SALES
-----------
      24500
```

5.2 Calculate Customer Total Spending

This function accepts Customer_ID as input and returns the total amount spent by that customer. It helps identify customer spending patterns.

```
SQL> CREATE OR REPLACE FUNCTION CUSTOMER_TOTAL_SPENDING (P_CUSTOMER_ID NUMBER) RETURN NUMBER AS V_TOTAL NUMBER(10,2); BEGIN SELECT NVL(SUM(TOTAL_AMOUNT),0)
INTO V_TOTAL FROM Orders WHERE CUSTOMER_ID = P_CUSTOMER_ID; RETURN V_TOTAL; END;
  2  /

Function created.

SQL> SELECT CUSTOMER_TOTAL_SPENDING(1) AS TOTAL_SPENDING FROM DUAL;

TOTAL_SPENDING
--------------
          5000
```

5.3 Calculate Product Revenue

This function accepts Product_ID as input and returns the total sales amount generated by that product. It helps analyse individual product revenue.

```
SQL> CREATE OR REPLACE FUNCTION CALCULATE_PRODUCT_REVENUE (P_ARTWORK_ID VARCHAR2) RETURN NUMBER AS V_REVENUE NUMBER(10,2); BEGIN SELECT NVL(SUM(D.QUANTITY *
 D.PRICE),0) INTO V_REVENUE FROM OrderDetails D WHERE D.ARTWORK_ID = TO_NUMBER(P_ARTWORK_ID); RETURN V_REVENUE; END;
  2  /

Function created.

SQL> SELECT CALCULATE_PRODUCT_REVENUE('101') AS TOTAL_REVENUE FROM DUAL;

TOTAL_REVENUE
-------------
        10000
```

6. Automate Repetitive Database Operations

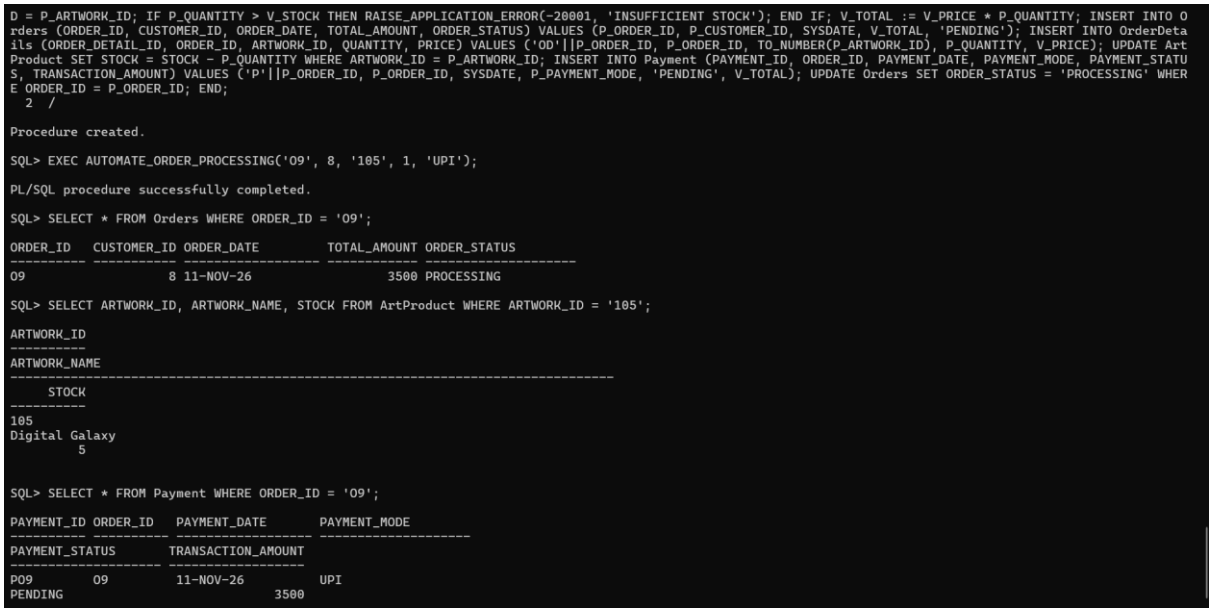
Database automation helps perform repeated operations automatically and reduces manual effort.

The following operations are implemented:

6.1 Order Processing Automation

When a customer places an order, the following operations are performed:

1. Insert order details.
2. Reduce product stock.
3. Create a payment record.
4. Update order status.



```
D = P_ARTWORK_ID; IF P_QUANTITY > V_STOCK THEN RAISE_APPLICATION_ERROR(-20001, 'INSUFFICIENT STOCK'); END IF; V_TOTAL := V_PRICE * P_QUANTITY; INSERT INTO O
rders (ORDER_ID, CUSTOMER_ID, ORDER_DATE, TOTAL_AMOUNT, ORDER_STATUS) VALUES (P_ORDER_ID, P_CUSTOMER_ID, SYSDATE, V_TOTAL, 'PENDING'); INSERT INTO OrderDeta
ils (ORDER_DETAIL_ID, ORDER_ID, ARTWORK_ID, QUANTITY, PRICE) VALUES ('OD'||P_ORDER_ID, P_ORDER_ID, TO_NUMBER(P_ARTWORK_ID), P_QUANTITY, V_PRICE); UPDATE Art
Product SET STOCK = STOCK - P_QUANTITY WHERE ARTWORK_ID = P_ARTWORK_ID; INSERT INTO Payment (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE, PAYMENT_STATU
S, TRANSACTION_AMOUNT) VALUES ('P'||P_ORDER_ID, P_ORDER_ID, SYSDATE, P_PAYMENT_MODE, 'PENDING', V_TOTAL); UPDATE Orders SET ORDER_STATUS = 'PROCESSING' WHER
E ORDER_ID = P_ORDER_ID; END;
  2  /

Procedure created.

SQL> EXEC AUTOMATE_ORDER_PROCESSING('O9', 8, '105', 1, 'UPI');

PL/SQL procedure successfully completed.

SQL> SELECT * FROM Orders WHERE ORDER_ID = 'O9';

ORDER_ID   CUSTOMER_ID ORDER_DATE         TOTAL_AMOUNT ORDER_STATUS
---------- ----------- ------------------ ------------ --------------------
O9                   8 11-NOV-26                  3500 PROCESSING

SQL> SELECT ARTWORK_ID, ARTWORK_NAME, STOCK FROM ArtProduct WHERE ARTWORK_ID = '105';

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
     STOCK
----------
105
Digital Galaxy
         5


SQL> SELECT * FROM Payment WHERE ORDER_ID = 'O9';

PAYMENT_ID ORDER_ID   PAYMENT_DATE       PAYMENT_MODE
---------- ---------- ------------------ --------------------
PAYMENT_STATUS       TRANSACTION_AMOUNT
-------------------- ------------------
PO9        O9         11-NOV-26          UPI
PENDING                          3500
```

This automation helps manage the order processing workflow efficiently.

6.2 Inventory Update Automation

Inventory automation helps maintain accurate stock information. The following operations are performed:

- Increase stock after new supply.
- Reduce stock after purchase.
- Identify low-stock products.

This helps maintain proper inventory records and monitor product availability.

```
SQL> CREATE OR REPLACE PROCEDURE UPDATE_INVENTORY (P_ARTWORK_ID VARCHAR2, P_QUANTITY NUMBER, P_OPERATION VARCHAR2) AS BEGIN IF UPPER(P_OPERATION) = 'ADD' TH
EN UPDATE ArtProduct SET STOCK = STOCK + P_QUANTITY WHERE ARTWORK_ID = P_ARTWORK_ID; ELSIF UPPER(P_OPERATION) = 'REMOVE' THEN UPDATE ArtProduct SET STOCK =
STOCK - P_QUANTITY WHERE ARTWORK_ID = P_ARTWORK_ID AND STOCK >= P_QUANTITY; IF SQL%ROWCOUNT = 0 THEN RAISE_APPLICATION_ERROR(-20002, 'INSUFFICIENT STOCK OR
INVALID ARTWORK ID'); END IF; ELSE RAISE_APPLICATION_ERROR(-20003, 'INVALID OPERATION'); END IF; END;
  2  /

Procedure created.

SQL> EXEC UPDATE_INVENTORY('105', 5, 'ADD');

PL/SQL procedure successfully completed.

SQL> EXEC UPDATE_INVENTORY('105', 2, 'REMOVE');

PL/SQL procedure successfully completed.

SQL> SELECT ARTWORK_ID, ARTWORK_NAME, STOCK FROM ArtProduct WHERE ARTWORK_ID = '105';

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
     STOCK
----------
105
Digital Galaxy
         8


SQL> SELECT ARTWORK_ID, ARTWORK_NAME, STOCK FROM ArtProduct WHERE STOCK <= 2;

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
     STOCK
----------
102
Colors of Nature
         2
```

6.3 Report Generation Automation

Report generation automation helps prepare frequently required business reports. The following reports are generated:

- Daily sales report.
- Monthly revenue report.
- Customer purchase report.

```
SQL> CREATE OR REPLACE VIEW DAILY_SALES_REPORT AS SELECT TRUNC(ORDER_DATE) AS SALES_DATE, COUNT(ORDER_ID) AS TOTAL_ORDERS, SUM(TOTAL_AMOUNT) AS TOTAL_REVENU
E FROM Orders GROUP BY TRUNC(ORDER_DATE);

View created.

SQL> SELECT * FROM DAILY_SALES_REPORT;

SALES_DATE         TOTAL_ORDERS TOTAL_REVENUE
------------------ ------------ -------------
24-SEP-26                     1          2800
21-SEP-26                     2          7000
23-SEP-26                     1          4000
22-SEP-26                     2          5700
11-NOV-26                     2          8500
20-SEP-26                     1          5000

6 rows selected.

SQL> CREATE OR REPLACE VIEW MONTHLY_REVENUE_REPORT AS SELECT TO_CHAR(ORDER_DATE, 'YYYY-MM') AS SALES_MONTH, COUNT(ORDER_ID) AS TOTAL_ORDERS, SUM(TOTAL_AMOUN
T) AS TOTAL_REVENUE FROM Orders GROUP BY TO_CHAR(ORDER_DATE, 'YYYY-MM');

View created.

SQL> SELECT * FROM MONTHLY_REVENUE_REPORT;

SALES_M TOTAL_ORDERS TOTAL_REVENUE
------- ------------ -------------
2026-09            7         24500
2026-11            2          8500
```

```
SQL> CREATE OR REPLACE VIEW CUSTOMER_PURCHASE_REPORT AS SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS, NVL(SUM(O.TOTAL_AMOUNT),0)
 AS TOTAL_SPENDING FROM Customer C LEFT JOIN Orders O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME;

View created.

SQL> SELECT * FROM CUSTOMER_PURCHASE_REPORT;

CUSTOMER_ID CUSTOMER_NAME                 TOTAL_ORDERS TOTAL_SPENDING
----------- ----------------------------- ------------ --------------
          4 SanjuSamson                              1           2200
         10 Jadeja                                   0              0
          8 Rohit Sharma                             2           8500
          1 Dhoni                                    1           5000
          6 Hardhik Pandya                           1           4000
          3 Ashwin                                   1           1800
          5 Shreyas Iyer                             1           3500
          2 Virat                                    1           5200
          7 Ruturaj Gaikwad                          1           2800
          9 Raina                                    0              0

10 rows selected.
```

These reports help analyse sales, revenue, and customer purchase information.

8. Result

The database automation features using Views, Stored Procedures, and Functions are implemented in the ArtHub database system. Repetitive database operations are organized and automated, and the created database objects are tested for correct execution and accurate results.

9. Conclusion

Thus, database automation using Views, Stored Procedures, and Functions is studied and implemented successfully. These features simplify frequently used reports, automate order processing, perform sales calculations, and reduce repetitive SQL coding in the ArtHub database system.